圖與二元搜尋樹的實作與走訪演算法分析：基於 Python 的物件導向設計

Implementation and Traversal Algorithm Analysis of Graph and BST

方柏森
1113405019
資訊工程學系 (Dept. of CSIE)
國立澎湖科技大學 (NPU)
Penghu, Taiwan
a32142166@gmail.com

***Abstract*—本研究旨在透過 Python 語言實作計算機科學中核心的非線性資料結構：「圖」(Graph) 與「二元搜尋樹」(Binary Search Tree, BST)。實驗中基於物件導向設計，詳細定義了圖的鄰接串列結構與樹的節點指標，並完整實作了多種走訪演算法。本研究深入探討了深度優先搜尋 (DFS) 中遞迴與顯式堆疊 (Explicit Stack) 的差異，以及廣度優先搜尋 (BFS) 在無權重圖中最短路徑的應用。針對 BST，則實作並驗證了前序、中序、後序與層序走訪。透過實證測試與微秒級別的效能追蹤，我們分析了不同走訪策略的特性。本專案的完整程式碼與 Markdown 說明文件已上傳至 GitHub，專案庫連結為：https://github.com/Eason0101-a/Algorithms_Assignment-5-2026-3-17。**

***Index Terms*—Graph, Binary Search Tree, Depth-First Search, Breadth-First Search, Data Structures, Python.**

I. 簡介 (INTRODUCTION)

在電腦科學領域，衡量資料管理與搜尋效率的核心標準之一便是資料結構的選用。相較於線性資料結構，圖結構與樹狀結構能夠更自然地表達資料間的階層與網路關係。二元搜尋樹結合了陣列的二分搜尋效率與鏈結串列的動態增刪優勢；而圖結構則能解決複雜的多對多關聯問題。為了深刻理解這兩者在面對節點動態操作與走訪時的表現，本實驗完整實作了 Graph 與 BST 模型，並透過實際的編碼與時間複雜度測量來觀察其內部行為。

II. 方法論 (METHODOLOGY)

本實驗基於物件導向設計 (OOP) 實作了兩大核心結構：

A. 圖的表示與走訪策略

本研究採用鄰接串列 (Adjacency List) 來儲存圖。走訪演算法分為兩種核心策略：

- **DFS**：優先深入探索單一分支。本研究同時探討了「系統呼叫疊 (Call Stack)」的遞迴實作，以及利用陣列模擬「顯式堆疊 (Explicit Stack)」的迭代實作。
- **BFS**：逐層向外探索。其特性使其非常適合用於尋找無權重圖的「最短路徑」(Shortest Path)。

兩者的時間複雜度理論值皆為 $O(V+E)$。

B. 二元搜尋樹核心性質

對於樹中的任意節點，其左子樹的所有節點值必定小於該節點值；其右子樹的所有節點值必定大於該節點值。此性質確保了中序走訪 (Inorder Traversal) 必定能輸出嚴格遞增的排序數列。

III. 實作細節 (IMPLEMENTATION DETAILS)

本系統完全使用 Python 撰寫，整合了以下核心模組：

- 圖進階操作：除了基本的 `dfs()` 與 `bfs()`，新增了 `dfs_stack()` 以迴避極端深度下的遞迴限制 (RecursionError)；並實作了 `shortest_path()` 方法，利用字典 `parent` 紀錄來源節點以回溯最短路徑。
- 樹狀走訪與管理：實作了 `insert()` 與 `build_from_list()`。走訪部分涵蓋了深度優先的前序、中序、後序，以及基於 `deque` 實作的層序走訪 (Level-order)，並設計了 `height()` 計算樹高。
- 效能評測：引入 Python 內建的 `time.perf_counter_ns()` 進行奈秒 (Nanosecond) 等級的效能精準測量，以比較不同走訪的執行開銷。

IV. 實驗結果 (EXPERIMENTAL RESULTS)

編譯並執行測試模組後，終端機輸出結果顯示兩大資料結構皆能正確運作。我們設計了「圖與樹走訪效能測試」以及「進階 Graph + BST 展示」兩個測試階段。

如圖 1 的終端機輸出所示，實驗證實：1) 走訪與效能：在針對樹狀圖形進行 DFS 與 BFS 時，`dfs_stack` 與 `bfs` 皆成功完成走訪，並印出奈秒級別的耗時，驗證了 $O(V+E)$ 的時間複雜度在小規模數據下的極速表現。2) 最短路徑：在有環的圖結構中，`shortest_path(2, 1)` 成功回傳了經過頂點 0 的最短路徑 `[2, 0, 1]`。3) **BST** 性質驗證：`inorder_traversal()` 確實輸出了 `[2, 5, 7, 10, 12, 15, 20]` 嚴格遞增的數列，且 `search(12)` 與 `search(99)` 皆正確回傳 True 與 False。

V. 討論 (DISCUSSION)

實測結果高度吻合理論預期。在 DFS 的實作探討中，雖然遞迴展現了極高的直觀性，但 `dfs_stack()` 利用顯式堆疊結合 `reversed()` 反向推入鄰居節點，不僅完美還原了遞迴的走訪順序，更消除了深度限制的隱患。而在 BFS 方面，透過字典追蹤父節點來反推最短路徑的技巧，

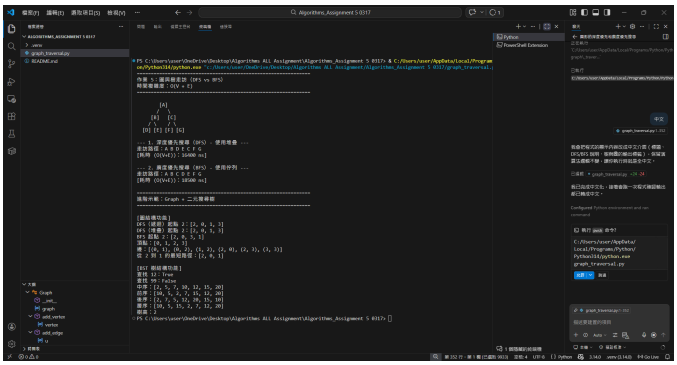

Fig. 1. 作業 5 測試程式執行結果截圖 (請確保同資料夾下有 figure.png 檔案)

展現了其在無權重網路路由中的實用價值。未來可進一步將圖的邊賦予權重，並引入 Dijkstra 等演算法進行擴展。

VI. 結論 (CONCLUSION)

本實驗成功展示了圖 (Graph) 與二元搜尋樹 (BST) 的內部結構與動態操作邏輯。透過 Python 實證觀察與效能測量，我們不僅驗證了 DFS 與 BFS 在走訪順序上的根本差異，更確認了各種走訪策略在不同應用場景（如排序、最短路徑尋找）下的優勢。這為後續進階的演算法開發奠定了堅實的基礎。

References

[1] 黃祥睿, “Lecture 2 Tree ,” 國立澎湖科技大學, 2026.

[2] Thomas H. Cormen, et al., “Introduction to Algorithms,” 3rd Edition, MIT Press.